FELIPE SILVA REZENDE
PEDRO ELIAS PRIBANIC FONSECA
GABRIELL BARRETO SANTIAGO DOS SANTOS

PROTEÇÃO, INTEGRIDADE E AUTENTICIDADE DE DOCUMENTOS DIGITAIS:

experimentos com AES-GCM, assinaturas RSA e certificados X.509 no caso XYZW Tecnologia

Relatório técnico apresentado à disciplina MATC99 – Criptografia Aplicada, do Instituto de Computação da Universidade Federal da Bahia, como requisito parcial de avaliação do Trabalho 01.

Professor: Joselito Júnior.

RESUMO

Este relatório avalia mecanismos criptográficos para proteger os documentos da empresa fictícia XYZW Tecnologia, diante de suspeitas de adulteração e de certificados não confiáveis. Uma proposta comercial foi cifrada com AES-256-GCM, que recuperou o texto com a chave correta e recusou a chave incorreta e a alteração de um único bit. Em seguida, o documento foi assinado com RSA-2048 e SHA-256 sob uma autoridade certificadora própria: o documento íntegro com certificado confiável foi aceito (caso A), o documento alterado após a assinatura foi rejeitado (caso B) e a assinatura válida emitida por autoridade não confiável também foi rejeitada (caso C). Os testes foram automatizados em Python e shell e confirmados com o OpenSSL.

Palavras-chave: AES-GCM; assinatura digital; certificado X.509; cadeia de confiança.

SALVADOR
2026

1 INTRODUÇÃO

A XYZW Tecnologia precisa garantir a confidencialidade, a integridade e a autenticidade dos documentos que compartilha com parceiros, após suspeitas de adulteração e de uso de certificados não confiáveis. Este trabalho testa, com arquivos fictícios, a cifração com AES-GCM e a assinatura digital com certificados X.509, usando Python e OpenSSL. As instruções de reprodução estão no `README.md` do repositório.

2 FUNDAMENTAÇÃO TEÓRICA

O AES é uma cifra de bloco simétrica de 128 bits; com chave de 256 bits, executa 14 rodadas (NIST, 2001). Modos como CBC e CTR oferecem só confidencialidade e são maleáveis: alterar bits do criptograma altera o texto decifrado sem que isso seja percebido. O GCM resolve esse problema ao combinar o modo contador com um autenticador (GHASH), formando uma cifra autenticada (AEAD) que gera, junto com o criptograma, uma etiqueta (*tag*) de 128 bits. Na decifração a tag é recalculada e, se não conferir, nenhum texto é liberado. A regra crítica do GCM é nunca repetir o nonce com a mesma chave, pois a repetição expõe os textos e permite forjar tags (DWORKIN, 2007).

Uma função de hash, como o SHA-256, gera um resumo de tamanho fixo resistente a colisões, e a mudança de um único caractere altera completamente o resumo (efeito avalanche) (NIST, 2015). Na assinatura digital, o emissor transforma o hash do documento com sua chave privada e qualquer pessoa verifica com a chave pública; o sucesso prova que o conteúdo não mudou e que foi assinado por quem detém a chave, mas não esconde o conteúdo. No RSA, recomenda-se o preenchimento PSS para assinatura, sendo o PKCS#1 v1.5 aceito, porém legado (MORIARTY *et al.*, 2016).

A assinatura comprova a posse de uma chave privada, mas não a quem ela pertence. O certificado X.509 associa a chave pública a uma identidade e é assinado por uma Autoridade Certificadora (CA). Validar a cadeia significa subir do certificado do signatário até uma âncora de confiança previamente instalada, conferindo assinatura, validade e extensões em cada elo (COOPER *et al.*, 2008). Por fim, a proteção depende da gestão das chaves: gerá-las com um gerador aleatório seguro, armazená-las, distribuí-las, rotacioná-las e revogá-las de forma controlada (BARKER, 2020; JÚNIOR, 2026).

3 METODOLOGIA

3.1 Ambiente e roteiro de testes

Foram desenvolvidos dois programas em Python, `criptografia.py` e `assinatura.py`, com interfaces de menu em linha de comando que seguem o mesmo padrão, mas com jornadas diferentes. Pela interface, o usuário escolhe o cenário de teste, informa o arquivo a proces-

sar e acompanha os arquivos gerados; chaves, nonces e certificados são gerados automaticamente. O arquivo `documento.txt`, uma proposta comercial fictícia de 572 bytes da XYZW para a "Empresa Parceira ABC", no valor de R$ 15.000,00, simula o cenário da atividade.

Para tornar os experimentos reproduzíveis, o roteiro `executar_testes.sh` cria uma área de testes limpa, alimenta os menus com as mesmas opções que um usuário digitaria, grava cada saída como evidência em `execucao_testes/evidencias/` e confere automaticamente o resultado esperado. As conclusões do Python são confirmadas por um caminho independente com o OpenSSL (`dgst -verify` e `verify`), evitando que um defeito do programa gere um falso positivo. O ambiente foi Windows 11 com Git Bash, Python 3.12.3, *cryptography* 50.0.2 (PYCA, 2026) e OpenSSL 1.1.1k (OPENSSL, 2021).

3.2 Programa de criptografia (criptografia.py)

O menu oferece: 1 – selecionar e criptografar documento; 2 – descriptografar com a chave correta; 3 – testar chave incorreta; 4 – alterar documento criptografado; 5 – testar detecção de alteração; 0 – sair. Assim, segue-se o caminho feliz da empresa sem lidar com a complexidade do processo e executam-se os casos de teste do método. A chave AES de 256 bits vem de `AESGCM.generate_key(bit_length=256)` e o nonce de 12 bytes de `os.urandom(12)`, ambos do gerador aleatório seguro do sistema. A chamada `AESGCM(chave).encrypt(nonce, documento, None)` devolve o criptograma seguido da tag de 16 bytes. Cada documento ganha uma pasta em `arquivos_criptografados/` com o criptograma (`.aes`), a chave, o nonce e o nome original. Como cada cifração gera chave e nonce próprios, a decifração usa os arquivos da pasta escolhida, e o par (chave, nonce) nunca se repete. No teste de chave incorreta, uma nova chave de 256 bits é gerada; na alteração, apenas o bit menos significativo do primeiro byte do criptograma é invertido (`dados[0] ^= 1`) e o resultado é salvo em `<nome>_adulterado.aes`, que o teste de detecção prioriza.

3.3 Programa de assinatura digital (assinatura.py)

O menu oferece: 1 – selecionar e assinar documento (autoridade XYZW); 2 – verificar os três cenários; 3 – alterar o próprio documento; 4 – assinar por autoridade desconhecida; 0 – sair. O documento escolhido é copiado para uma pasta em `assinaturas_digitais/`, com uma cópia de segurança (`.save.txt`). A função `criar_ca` gera um par RSA de 2048 bits e um certificado autoassinado "CN=CA_XYZW", com número de série aleatório, validade de 365 dias e extensão `BasicConstraints(ca=True)`, assinado com SHA-256. A função `emitir_certificado` repete o processo para "CN=Signatario XYZW", com `ca=False`, 180 dias e assinatura pela chave privada da CA, formando a cadeia CA_XYZW → Signatario XYZW. Chaves e certificados ficam na pasta e, então, o documento é assinado com `sign(conteudo, padding.PKCS1v15(), hashes.SHA256())`, gerando `assinatura_A.bin` (256 bytes).

A verificação separa duas perguntas: a criptográfica, em que a chave pública do certificado confere a assinatura sobre o conteúdo atual, e a de confiança, com `openssl verify -CAfile ca_confiavel.pem`, que aceita só a CA_XYZW como âncora. A opção 3 sobrescreve o documento mantendo a assinatura original. A opção 4 cria a "CN=CA_DESCONHECIDA", emite "CN=Signatario Externo" e assina a cópia original, isolando o caso C da adulteração do B.

4 EXPERIMENTOS

Os experimentos seguiram a sequência do Quadro 1, executada pelo roteiro de testes. Em cada etapa, o roteiro registra a saída do programa e confirma o resultado esperado; nas etapas de assinatura, repete a verificação com o OpenSSL.

Quadro 1 – Roteiro dos experimentos

Experimento	Opção do menu	O que se verifica
I.2 Cifrar	`criptografia.py`, 1	Chave de 32 bytes, nonce de 12 bytes, criptograma = texto + 16 bytes de tag, texto ausente do criptograma
I.3 Chave correta	`criptografia.py`, 5 e 2	Integridade confirmada no criptograma intacto; arquivo recuperado idêntico ao original (SHA-256 e `cmp`)
I.4 Chave incorreta	`criptografia.py`, 3	Decifração recusada, sem liberar texto
I.4 Criptograma alterado	`criptografia.py`, 4 e 5	Apenas 1 byte difere (`cmp -l`); decifração recusada
Caso A	`assinatura.py`, 1 e 2	Assinatura válida e cadeia confiável; confirmação com `dgst -verify` e `verify`
Caso B	`assinatura.py`, 3 e 2	Valor alterado de R$ 15.000,00 para R$ 150.000,00; assinatura inválida com cadeia confiável
Caso C	`assinatura.py`, 4 e 2	Assinatura válida, mas cadeia não confiável contra a CA_XYZW

Fonte: elaborado pelos autores (2026).

5 RESULTADOS

O roteiro aprovou as 44 verificações previstas. Chaves, nonces, séries e assinaturas são aleatórios e mudam a cada execução; os valores citados são os da execução registrada nas evidências.

5.1 Parte I: proteção e integridade com AES-GCM

O documento original tem SHA-256 `69586d17…4593fa23d`. O criptograma tem 588 bytes, ou seja, os 572 do texto mais os 16 da tag, começa por `46 6c aa 2f ec 8c…` e não contém a sequência "XYZW TECNOLOGIA". O criptograma intacto passou na verificação de integridade, e o arquivo recuperado com a chave correta tem o mesmo SHA-256 do original. Com uma chave incorreta, o programa informou "Falha! Chave incorreta detectada.", pois a

biblioteca lançou a exceção `InvalidTag`. Após a alteração, `cmp -l` mostrou apenas o byte 1 modificado, de 0x46 para 0x47, e o programa informou “Falha! Adulteração detectada pelo AES-GCM.”.

Como o GCM cifra em modo contador, inverter o primeiro bit do criptograma inverteria o mesmo bit do texto decifrado, transformando “X” (0x58) em “Y” (0x59): uma adulteração silenciosa em CTR ou CBC. No GCM, a tag recalculada diverge e a decifração inteira é recusada. Chave errada e criptograma adulterado produzem o mesmo erro, pois a tag só indica que o par (chave, dados) não é autêntico. Ficam garantidas a confidencialidade, a integridade e a autenticidade da origem em sentido simétrico, já que só quem tem a chave gera uma tag válida. Não há não repúdio, pois a chave é compartilhada.

5.2 Parte II: casos A, B e C

Todos os certificados usam RSA-2048 e `sha256WithRSAEncryption` e estão no Quadro 2.

Quadro 2 – Certificados gerados

Certificado	Emissor	Tipo e validade	Impressão digital SHA-256
CN=CA_XYZW	CN=CA_XYZW	CA raiz; 365 dias	`CD:5E:F1:7A:A9:10…`
CN=Signatario XYZW	CN=CA_XYZW	Entidade final; 180 dias	`72:06:D8:7D:37:E1…`
CN=CA_DESCONHECIDA	CN=CA_DESCONHECIDA	CA raiz; 365 dias	`19:B7:75:39:5D:D3…`
CN=Signatario Externo	CN=CA_DESCONHECIDA	Entidade final; 180 dias	`0F:60:84:5C:ED:86…`

Fonte: elaborado pelos autores (2026), com `openssl x509 -fingerprint -sha256`.

Caso A. O programa informou “Assinatura valida: True” e “Cadeia confiavel: True”, e o OpenSSL confirmou com “Verified OK” e “signatario_A.pem: OK”. A chave pública do Signatario XYZW reproduz o hash do documento a partir da assinatura, e o certificado foi assinado pela CA_XYZW, âncora da empresa. **Decisão: aceitar**, pois o documento é íntegro e o signatário é identificado por uma CA confiável.

Caso B. O `diff` mostra só a linha 14 alterada, mas o SHA-256 mudou por completo, de `69586d17…4593fa23d` para `2e9267c3…28cec34c1` (efeito avalanche). O programa informou “Assinatura valida no arquivo atual: False” com a cadeia ainda “True”, e o OpenSSL retornou “Verification Failure” no arquivo alterado e “Verified OK” na cópia original. **Decisão: rejeitar**: o certificado é confiável, mas a assinatura não corresponde ao conteúdo, prova de alteração posterior à assinatura.

Caso C. O programa informou “Assinatura valida: True” e “Cadeia confiavel: False”, confirmados pelo OpenSSL na Figura 1.

Figura 1 – Verificação da assinatura C e da cadeia contra a âncora CA_XYZW

```
$ openssl dgst -sha256 -verify pub_C.pem -signature assinatura_C.bin documento.save.txt
Verified OK
$ openssl verify -CAfile ca_confiavel.pem signatario_C.pem
CN = Signatario Externo
error 20 at 0 depth lookup: unable to get local issuer certificate
error signatario_C.pem: verification failed
$ openssl verify -CAfile ca_desconhecida.pem signatario_C.pem
signatario_C.pem: OK
```

Fonte: elaborado pelos autores (2026), a partir de `execucao_testes/evidencias/C_openssl_*.log`.

O erro 20 indica que o emissor, CA_DESCONHECIDA, não está entre as âncoras confiáveis; a cadeia só fecharia se essa CA fosse instalada como confiável. **Decisão: rejeitar**: a matemática confere, mas não há como afirmar quem é o signatário, pois qualquer pessoa cria uma CA com o nome que quiser. O Quadro 3 consolida os casos.

Quadro 3 – Síntese da Parte II

Caso	Assinatura	Cadeia	Evidência principal	Decisão
A	Válida	Confiável	Verified OK; signatario_A.pem: OK	Aceitar
B	Inválida	Confiável	Verification Failure; SHA-256 alterado	Rejeitar
C	Válida	Não confiável	Verified OK; error 20 no `verify`	Rejeitar

Fonte: elaborado pelos autores (2026).

6 ANÁLISE DO INCIDENTE

As suspeitas da XYZW correspondem aos casos B e C. A adulteração, como a mudança do valor de uma proposta, é detectada pela assinatura, desde que o destinatário a verifique e rejeite o documento quando ela falhar. O uso de certificados não confiáveis é um ataque à identidade: o atacante não quebra o RSA, apenas cria sua própria CA e conta que a vítima confie nela ou ignore o erro de cadeia. A causa provável do incidente, portanto, são falhas de processo, e não dos algoritmos. A regra que os experimentos sustentam é aceitar um documento só quando a assinatura é válida sobre o conteúdo recebido e a cadeia chega a uma âncora confiável, dentro da validade e sem revogação.

A implementação, didática, tem limitações a corrigir em produção: a chave AES fica em claro ao lado do criptograma, em vez de ser distribuída por criptografia híbrida ou guardada em um cofre de chaves (KMS ou HSM); as chaves privadas das CAs e dos signatários ficam sem senha na mesma pasta; a assinatura usa PKCS#1 v1.5 em vez de PSS; e os certificados não têm verificação de revogação nem carimbo de tempo.

Recomenda-se à XYZW: manter uma lista fechada de CAs confiáveis e conferir fora de banda a impressão digital SHA-256 de cada uma; adotar RSA-PSS, verificação de revogação e carimbo de tempo; assinar e depois cifrar documentos sigilosos com AES-GCM, com nonce único por chave; gerir chaves em KMS ou HSM, com rotação e revogação; e registrar como evidência de auditoria o hash, a assinatura, a impressão digital do certificado e o resultado de cada verificação.

7 CONCLUSÕES

Os experimentos confirmaram, com evidências reproduzíveis e conferidas por duas ferramentas, o comportamento esperado de cada mecanismo. O AES-256-GCM protegeu o sigilo da proposta, permitiu a recuperação exata com a chave correta e recusou a chave incorreta e a alteração de um único bit, fechando a lacuna de maleabilidade dos modos sem autenticação. A assinatura RSA com SHA-256 detectou a adulteração do valor (caso B), e o caso C mostrou que uma assinatura matematicamente válida não basta sem cadeia até uma âncora confiável. A proteção dos documentos da XYZW depende, assim, tanto dos algoritmos quanto da gestão de chaves e da política de confiança.

REFERÊNCIAS

BARKER, E. **Recommendation for key management**: part 1: general. Gaithersburg: National Institute of Standards and Technology, 2020. (NIST Special Publication 800-57 Part 1 Revision 5). DOI: 10.6028/NIST.SP.800-57pt1r5.

COOPER, D. *et al.* **Internet X.509 public key infrastructure certificate and certificate revocation list (CRL) profile**. [*S. l.*]: Internet Engineering Task Force, 2008. (RFC 5280). Disponível em: https://www.rfc-editor.org/rfc/rfc5280. Acesso em: 6 nov. 2026.

DWORKIN, M. **Recommendation for block cipher modes of operation**: Galois/Counter Mode (GCM) and GMAC. Gaithersburg: National Institute of Standards and Technology, 2007. (NIST Special Publication 800-38D). DOI: 10.6028/NIST.SP.800-38D.

JÚNIOR, Joselito. **Aula 03**: fundamentos de criptografia aplicada. Salvador: Universidade Federal da Bahia, 2026. Material de aula da disciplina MATC99.

MORIARTY, K. *et al.* **PKCS #1**: RSA cryptography specifications version 2.2. [*S. l.*]: Internet Engineering Task Force, 2016. (RFC 8017). Disponível em: https://www.rfc-editor.org/rfc/rfc8017. Acesso em: 6 nov. 2026.

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. **Advanced Encryption Standard (AES)**. Gaithersburg: NIST, 2001. (FIPS PUB 197, atualizado em 2023). DOI: 10.6028/NIST.FIPS.197-upd1.

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. **Secure Hash Standard (SHS)**. Gaithersburg: NIST, 2015. (FIPS PUB 180-4). DOI: 10.6028/NIST.FIPS.180-4.

OPENSSL. **OpenSSL 1.1.1 manual pages**: openssl-verify, openssl-dgst, openssl-x509. [*S. l.*]: OpenSSL Project, 2021. Disponível em: https://www.openssl.org/docs/man1.1.1/. Acesso em: 6 nov. 2026.

PYCA. **cryptography**: documentation. Versão 50.0.2. [*S. l.*]: Python Cryptographic Authority, 2026. Disponível em: https://cryptography.io/. Acesso em: 6 nov. 2026.